비콘 서비스

소개

비콘 서비스는 Oasis Core 네트워크에서 시간 관리와 랜덤성 제공을 담당하는 핵심 서비스입니다. 이 서비스는 에포크(epoch) 기반의 시간 시스템을 관리하고, VRF(Verifiable Random Function)를 통해 검증 가능한 랜덤 비콘을 생성합니다.

주요 특징

- **에포크 기반 시간 관리**: 블록 높이를 에포크 단위로 변환하여 시간 관리
- **검증 가능한 랜덤성**: VRF를 통한 예측 불가능하고 검증 가능한 랜덤 값 생성
- **결정론적 실행**: 모든 노드에서 동일한 에포크와 랜덤 값 보장
- **유연한 백엔드**: Insecure와 VRF 두 가지 백엔드 지원

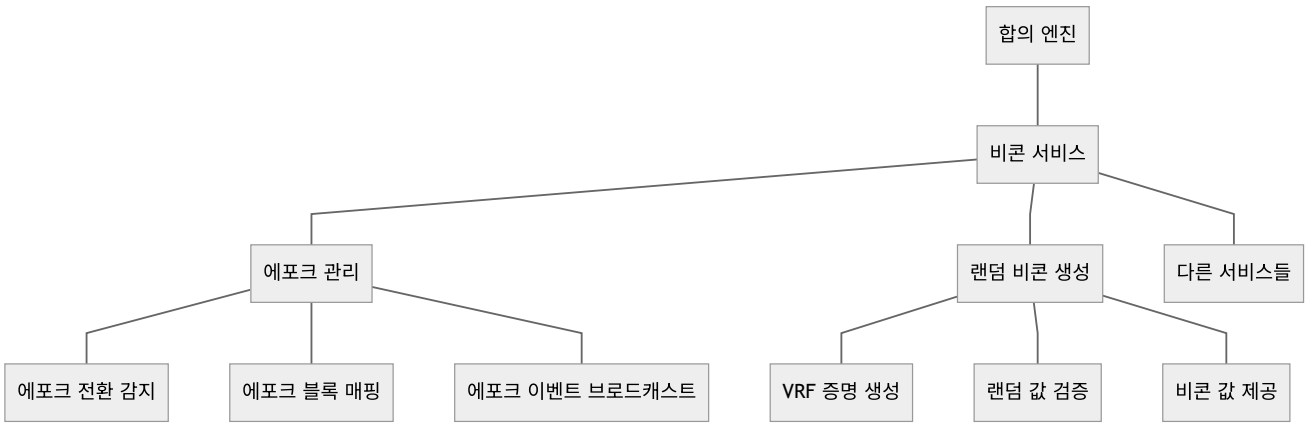
관련 프로세스

- **합의 프로세스**: 에포크 전환과 VRF 증명 트랜잭션 처리
- **스케줄러 서비스**: 에포크 기반 위원회 스케줄링
- **스테이킹 서비스**: 에포크 기반 보상 분배와 슬래싱
- **거버넌스 서비스**: 에포크 기반 제안 생명주기 관리

아키텍처

전체 구조

비콘 서비스는 시간 관리와 랜덤성 제공이라는 두 가지 핵심 기능을 제공합니다.



주요 컴포넌트

에포크 관리자

역할: 블록 높이를 에포크로 변환하고 에포크 전환을 관리

주요 기능:

- 현재 에포크 계산
- 에포크 전환 감지
- 에포크 블록 높이 매핑
- 에포크 이벤트 발생

VRF 백엔드

역할: 검증 가능한 랜덤 함수를 통한 랜덤 비콘 생성

주요 기능:

- VRF 증명 생성 및 검증
- 랜덤 비콘 값 계산
- VRF 상태 관리

- 증명 제출 스케줄링

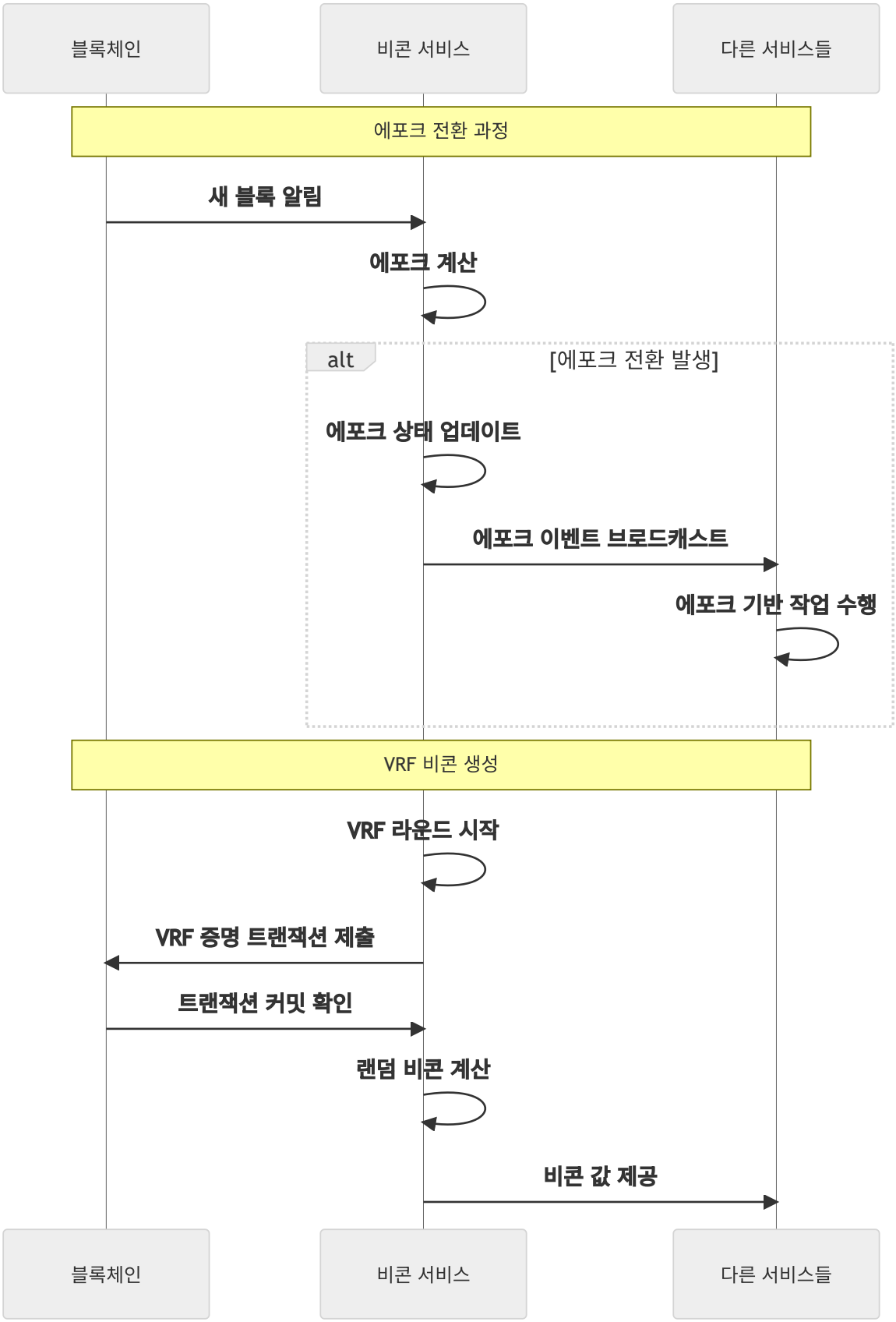
비콘 캐시

역할: 에포크와 비콘 값의 효율적인 조회를 위한 캐싱

주요 기능:

- 최근 에포크 정보 캐싱
- 비콘 값 캐싱
- LRU 기반 캐시 관리

에포크 시스템


블록체인
비콘 서비스
다른 서비스들
에포크 전환 과정
새 블록 알림
에포크 계산
alt
[에포크 전환 발생]
에포크 상태 업데이트
에포크 이벤트 브로드캐스트
에포크 기반 작업 수행
VRF 비콘 생성
VRF 라운드 시작
VRF 증명 트랜잭션 제출
트랜잭션 커밋 확인
랜덤 비콘 계산
비콘 값 제공
블록체인
비콘 서비스
다른 서비스들

랜덤 비콘 생성 메커니즘

VRF (Verifiable Random Function) 개요

VRF는 개인키를 가진 사용자만이 특정 입력에 대해 의사랜덤 출력과 그 출력이 올바르게 계산되었음을 증명하는 증명을 생성할 수 있는 암호학적 함수입니다.

VRF 비콘 생성 과정

1. **알파 값 생성**: 각 에포크마다 고유한 알파(α) 값 생성
2. **증명 생성**: 밸리데이터들이 VRF 개인키로 알파 값에 대한 증명 생성
3. **증명 수집**: 합의를 통해 VRF 증명들을 수집
4. **비콘 계산**: 수집된 증명들을 조합하여 최종 랜덤 비콘 값 계산

```
// VRF 상태 구조체
type VRFState struct {
// Epoch는 현재 VRF 라운드의 에포크입니다
Epoch EpochTime `json:"epoch"`
// Alpha는 VRF 입력 값입니다
Alpha []byte  `json:"alpha"`
// SubmitAfter는 증명 제출이 허용되는 최소 블록 높이입니다
SubmitAfter  int64  `json:"submit_after"`
// Pi는 각 노드의 VRF 증명들입니다
Pi  map[signature.PublicKey][]byte  `json:"pi,omitempty"`
}
```

VRF 증명 생성

```go
// VRF 증명 생성 예시
func generateVRFProof(vrfSigner signature.Signer, alpha []byte) ([]byte, error)
// VRF 증명 생성
proof, err := signature.Prove(vrfSigner, alpha)
if err != nil {
return nil, fmt.Errorf("VRF 증명 생성 실패: %w", err)
}
return proof.Proof[:], nil
}

// VRF 증명 검증
func verifyVRFProof(publicKey signature.PublicKey, alpha []byte, proof []byte)
// 증명 검증 및 출력 계산
output, err := signature.Verify(publicKey, alpha, proof)
if err != nil {
return nil, fmt.Errorf("VRF 증명 검증 실패: %w", err)
}
return output, nil
}
```

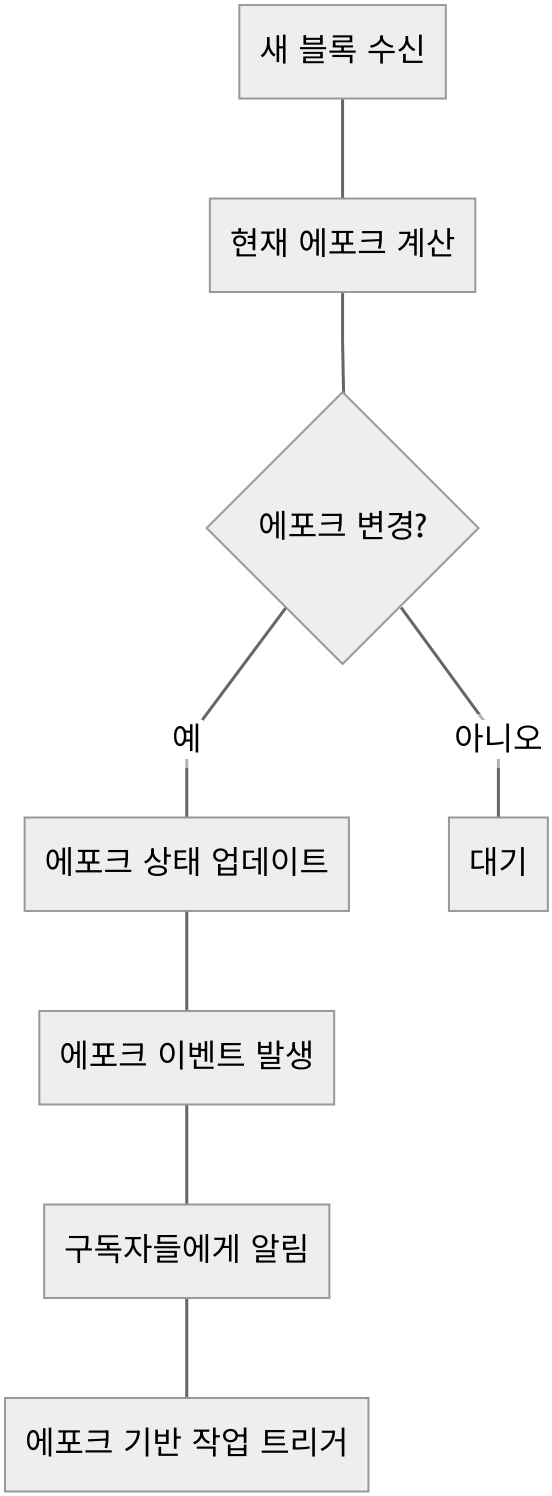
에포크 기반 시간 관리 시스템

에포크 계산

에포크는 블록 높이를 기반으로 계산되며, 설정된 간격(interval)에 따라 결정됩니다.

```
// 에포크 계산 공식
func calculateEpoch(baseEpoch EpochTime, baseHeight int64, currentHeight int6
if currentHeight < baseHeight {
return baseEpoch
}
blocksSinceBase := currentHeight - baseHeight
epochsSinceBase := blocksSinceBase / interval
return baseEpoch + EpochTime(epochsSinceBase)
}
```

에포크 전환 감지



에포크 이벤트 처리

```
// 에포크 이벤트 구조체
type EpochEvent struct {
Epoch EpochTime `json:"epoch,omitempty"`
}
```

```go
// 에포크 변경 감지 및 이벤트 발생
func (sc *ServiceClient) updateCachedEpoch(height int64, epoch EpochTime) bool
sc.Lock()
defer sc.Unlock()
sc.epoch = epoch
sc.epochCurrentBlock = height
// 에포크가 변경된 경우에만 이벤트 발생
if sc.epochLastNotified != epoch {
sc.logger.Debug("에포크 전환",
"이전_에포크", sc.epochLastNotified,
"현재_에포크", epoch,
"블록_높이", height,
)
sc.epochLastNotified = epoch
return true // 이벤트 발생 필요
}
return false
}
```

VRF 구현 설명

VRF 워커

VRF 워커는 VRF 증명을 자동으로 생성하고 제출하는 백그라운드 프로세스입니다.

```go
type vrfWorker struct {
parent *Worker
logger *logging.Logger
backend beacon.VRFBackend
txRetry *txRetry
enabled  bool
quitCh  chan  struct{}
}

// VRF 워커 메인 루프
func (w *vrfWorker) worker() {
// VRF 이벤트 구독
eventCh, eventSub, err := w.backend.WatchLatestVRFEvent(w.parent.ctx)
if  err != nil {
w.logger.Error("VRF 이벤트 구독 실패", "err", err)
return
}
defer  eventSub.Close()
for {
select {
case <-w.quitCh:
```

```go
return
case ev := <-eventCh:
// VRF 증명 생성 및 제출
w.handleVRFEvent(ev)
}
}
}
```

VRF 증명 제출 과정

1. **VRF 이벤트 수신**: 새로운 VRF 라운드 시작 알림
2. **상태 확인**: 현재 VRF 상태와 제출 조건 확인
3. **증명 생성**: VRF 개인키로 알파 값에 대한 증명 생성
4. **트랜잭션 생성**: VRF 증명을 포함한 트랜잭션 생성
5. **트랜잭션 제출**: 합의 네트워크에 트랜잭션 제출

```go
// VRF 증명 제출 처리
func (w *vrfWorker) handleVRFEvent(ev *beacon.VRFEvent) {
// VRF 상태 조회
vrfState, err := w.backend.GetVRFState(w.parent.ctx, consensus.HeightLatest)
if err != nil {
w.logger.Error("VRF 상태 조회 실패", "err", err)
return
}
```

```
// 제출 조건 확인
if !w.shouldSubmitProof(vrfState) {
return
}
// VRF 증명 생성
pi, err := signature.Prove(w.parent.identity.VRFSigner, vrfState.Alpha)
if  err != nil {
w.logger.Error("VRF 증명 생성 실패", "err", err)
return
}
// 트랜잭션 생성 및 제출
proofPayload := beacon.VRFProve{
Epoch: vrfState.Epoch,
Pi: pi.Proof[:],
}
tx := transaction.NewTransaction(0, nil, beacon.MethodVRFProve, proofPayload)
w.retrySubmitTx(tx, vrfState.Epoch)
}
```

API 및 인터페이스

비콘 백엔드 인터페이스

```go
// Backend는 비콘/시간 관리 구현을 위한 인터페이스입니다
type Backend interface {
// GetBaseEpoch는 기준 에포크를 반환합니다
GetBaseEpoch(context.Context) (EpochTime, error)
// GetEpoch는 지정된 블록 높이의 에포크 번호를 반환합니다
GetEpoch(context.Context, int64) (EpochTime, error)
// GetFutureEpoch는 예정된 미래 에포크 정보를 반환합니다
GetFutureEpoch(context.Context, int64) (*EpochTimeState, error)
// GetEpochBlock은 지정된 에포크의 시작 블록 높이를 반환합니다
GetEpochBlock(context.Context, EpochTime) (int64, error)
// WaitEpoch는 특정 에포크까지 대기합니다
WaitEpoch(ctx context.Context, epoch EpochTime) error
// WatchEpochs는 에포크 전환 이벤트 스트림을 반환합니다
WatchEpochs(ctx context.Context) (<-chan EpochTime, pubsub.ClosableSubscription,
// GetBeacon은 지정된 블록 높이의 랜덤 비콘을 반환합니다
GetBeacon(context.Context, int64) ([]byte, error)
// ConsensusParameters는 비콘 합의 파라미터를 반환합니다
ConsensusParameters(ctx context.Context, height  int64) (*ConsensusParameters, e
}
```

VRF 백엔드 인터페이스

```go
// VRFBackend는 VRF 관련 기능을 위한 확장 인터페이스입니다
type VRFBackend interface {
Backend
// GetVRFState는 지정된 높이의 VRF 상태를 반환합니다
GetVRFState(ctx context.Context, height  int64) (*VRFState, error)
// WatchLatestVRFEvent는 최신 VRF 이벤트 스트림을 반환합니다
WatchLatestVRFEvent(ctx context.Context) (<-chan *VRFEvent, *pubsub.Subscription
}
```

gRPC API

비콘 서비스 정의

```proto
service BeaconService {
// GetEpoch는 지정된 높이의 에포크를 조회합니다
rpc  GetEpoch(GetEpochRequest) returns (GetEpochResponse);
// GetEpochBlock은 지정된 에포크의 시작 블록을 조회합니다
rpc  GetEpochBlock(GetEpochBlockRequest) returns (GetEpochBlockResponse);
// GetBeacon은 지정된 높이의 랜덤 비콘을 조회합니다
rpc  GetBeacon(GetBeaconRequest) returns (GetBeaconResponse);
// WatchEpochs는 에포크 전환 이벤트를 스트리밍합니다
rpc  WatchEpochs(WatchEpochsRequest) returns (stream EpochEvent);
}
```

메시지 타입

```
message GetEpochRequest {
int64  height = 1; // 블록 높이 (0은 최신 블록)
}

message GetEpochResponse {
uint64  epoch = 1; // 에포크 번호
}

message GetBeaconRequest {
int64  height = 1; // 블록 높이
}

message GetBeaconResponse {
bytes  beacon = 1; // 랜덤 비콘 값 (32바이트)
}

message EpochEvent {
uint64  epoch = 1; // 새로운 에포크 번호
}
```

설정 및 구성

비콘 설정 구조

```yaml
beacon:
# 비콘 백엔드 종류 (insecure 또는 vrf)
backend: "vrf"
# 기준 에포크 (제네시스에서 설정)
base: 0
# Insecure 백엔드 설정
insecure_parameters:
interval: 10  # 에포크 간격 (블록 수)
# VRF 백엔드 설정
vrf_parameters:
interval: 600  # 에포크 간격 (블록 수, 약 1시간)
proof_delay: 10  # 증명 제출 지연 (블록 수)
alpha_high_qc_threshold: 0.67  # 알파 생성을 위한 QC 임계값
submission_window: 100  # 증명 제출 윈도우 (블록 수)
gas_costs:
vrf_prove: 1000  # VRF 증명 트랜잭션 가스 비용
```

주요 설정 옵션

| 옵션 이름 | 타입 | 기본값 | 설명 |

|----------|------|--------|------|

| backend | string | “vrf” | 비콘 백엔드 종류 |

| `base` | uint64 | 0 | 기준 에포크 |

| `insecure_parameters.interval` | int64 | 10 | Insecure 백엔드 에포크 간격 |

| `vrf_parameters.interval` | int64 | 600 | VRF 백엔드 에포크 간격 |

| `vrf_parameters.proof_delay` | int64 | 10 | VRF 증명 제출 지연 |

| `vrf_parameters.alpha_high_qc_threshold` | float64 | 0.67 | 알파 생성 QC 임계값 |

환경 변수

| 환경 변수 | 설명 | 예시 |

|----------|------|------|

| `OASIS_BEACON_BACKEND` | 비콘 백엔드 지정 | `vrf`, `insecure` |

| `OASIS_BEACON_VRF_INTERVAL` | VRF 에포크 간격 | `600` |

| `OASIS_BEACON_DEBUG_MOCK` | 테스트용 목 백엔드 활성화 | `true`, `false` |

코드 예제

기본 사용법

```go
package main


import (
"context"
"fmt"
"log"
"github.com/oasisprotocol/oasis-core/go/beacon/api"
```

```go
"github.com/oasisprotocol/oasis-core/go/consensus/api"
)

func main() {
// 합의 서비스 연결
consensus, err := api.NewConsensusClient("unix:/tmp/oasis-node-consensus.sock")
if err != nil {
log.Fatal("합의 클라이언트 생성 실패:", err)
}
// 비콘 백엔드 가져오기
beacon := consensus.Beacon()
ctx := context.Background()
// 현재 에포크 조회
currentEpoch, err := beacon.GetEpoch(ctx, api.HeightLatest)
if err != nil {
log.Fatal("현재 에포크 조회 실패:", err)
}
fmt.Printf("현재 에포크: %d\n", currentEpoch)
// 현재 랜덤 비콘 조회
randomBeacon, err := beacon.GetBeacon(ctx, api.HeightLatest)
if err != nil {
log.Fatal("랜덤 비콘 조회 실패:", err)
}
```

```
fmt.Printf("랜덤 비콘: %x\n", randomBeacon)
}
```

에포크 전환 모니터링

```
// 에포크 전환 이벤트 모니터링
func watchEpochTransitions(beacon api.Backend) error {
ctx := context.Background()
// 에포크 이벤트 구독
epochCh, sub, err := beacon.WatchEpochs(ctx)
if err != nil {
return fmt.Errorf("에포크 이벤트 구독 실패: %w", err)
}
defer sub.Close()
fmt.Println("에포크 전환을 모니터링합니다...")
for {
select {
case epoch := <-epochCh:
fmt.Printf("에포크 전환: %d\n", epoch)
// 에포크 전환 시 수행할 작업
if err := handleEpochTransition(beacon, epoch); err != nil {
log.Printf("에포크 전환 처리 실패: %v", err)
}
```

```go
case <-ctx.Done():
return ctx.Err()
}
}
}

func handleEpochTransition(beacon api.Backend, epoch api.EpochTime) error {
ctx := context.Background()
// 새 에포크의 시작 블록 높이 조회
startHeight, err := beacon.GetEpochBlock(ctx, epoch)
if err != nil {
return fmt.Errorf("에포크 시작 블록 조회 실패: %w", err)
}
fmt.Printf("에포크 %d는 블록 %d에서 시작됩니다\n", epoch, startHeight)
// 에포크별 작업 수행 (예: 위원회 업데이트, 보상 분배 등)
return nil
}
```

VRF 증명 생성

```go
// VRF 증명 생성 및 제출
func submitVRFProof(consensus api.Backend, vrfSigner signature.Signer) error {
ctx := context.Background()
```

```go
beacon := consensus.Beacon()
// VRF 백엔드로 캐스팅
vrfBackend, ok := beacon.(api.VRFBackend)
if !ok {
return fmt.Errorf("VRF 백엔드가 아닙니다")
}
// 현재 VRF 상태 조회
vrfState, err := vrfBackend.GetVRFState(ctx, api.HeightLatest)
if err != nil {
return fmt.Errorf("VRF 상태 조회 실패: %w", err)
}
if vrfState == nil {
return fmt.Errorf("VRF 상태가 없습니다")
}
// VRF 증명 생성
proof, err := signature.Prove(vrfSigner, vrfState.Alpha)
if err != nil {
return fmt.Errorf("VRF 증명 생성 실패: %w", err)
}
// VRF 증명 트랜잭션 생성
proofPayload := &api.VRFProve{
Epoch: vrfState.Epoch,
Pi: proof.Proof[:],
```

```go
}
tx := &transaction.Transaction{
Nonce: 0,
Fee: transaction.NewFee(1000, 0),
Method: api.MethodVRFProve,
Body: proofPayload,
}
// 트랜잭션 서명 및 제출
signedTx, err := transaction.Sign(vrfSigner, tx)
if err != nil {
return fmt.Errorf("트랜잭션 서명 실패: %w", err)
}
if err := consensus.SubmitTx(ctx, signedTx); err != nil {
return fmt.Errorf("VRF 증명 제출 실패: %w", err)
}
fmt.Printf("VRF 증명이 에포크 %d에 대해 제출되었습니다\n", vrfState.Epoch)
return nil
}
```

CLI 사용 예제

```bash
# 현재 에포크 조회
oasis-node beacon show-epoch
```

```
# 특정 높이의 에포크 조회
oasis-node beacon show-epoch --height 12345

# 현재 랜덤 비콘 조회
oasis-node beacon show-beacon

# 에포크 파라미터 조회
oasis-node beacon show-parameters

# VRF 상태 조회 (VRF 백엔드인 경우)
oasis-node beacon show-vrf-state

# 에포크 전환 대기
oasis-node beacon wait-epoch --epoch 100
```

모니터링 및 메트릭

주요 메트릭

메트릭 이름	타입	설명
`oasis_beacon_epochs_total`	Counter	총 에포크 전환 수
`oasis_beacon_epoch_transition_lag_blocks`	Histogram	에포크 전환 지연 (블록 수)

| `oasis_beacon_vrf_proofs_total` | Counter | 제출된 VRF 증명 수 |

| `oasis_beacon_vrf_proof_generation_duration_seconds` | Histogram | VRF 증명 생성 시간 |

| `oasis_beacon_current_epoch` | Gauge | 현재 에포크 번호 |

| `oasis_beacon_epoch_interval_blocks` | Gauge | 에포크 간격 (블록 수) |

헬스 체크

```
# 비콘 서비스 상태 확인
curl http://localhost:9001/beacon/status

# 응답 예시
{
"backend": "vrf",
"current_epoch": 1234,
"current_height": 123456,
"epoch_interval": 600,
"base_epoch": 0,
"vrf_enabled": true,
"last_beacon": "a1b2c3d4e5f6789..."
}
```

로그 모니터링

```
# 비콘 관련 로그 확인
```

```
journalctl -u oasis-node -f | grep beacon


# 에포크 전환 로그 확인
journalctl -u oasis-node | grep "epoch transition"


# VRF 관련 로그 확인
journalctl -u oasis-node | grep "VRF"


# 에러 로그 필터링
journalctl -u oasis-node --since "1 hour ago" | grep ERROR | grep beacon
```

문제 해결

일반적인 문제

문제 1: 에포크가 전환되지 않음

증상: 예상된 시간에 에포크가 전환되지 않음

원인:

- 블록 생성 지연
- 에포크 간격 설정 오류
- 비콘 서비스 동기화 문제

해결 방법:

1. 현재 블록 높이와 에포크 간격 확인

```
oasis-node consensus status

oasis-node beacon show-parameters
```

2. 에포크 계산 검증

```
# 현재 에포크 = 기준_에포크 + (현재_높이 - 기준_높이) / 간격
```

3. 비콘 서비스 재시작

문제 2: VRF 증명 제출 실패

증상: VRF 증명이 제출되지 않거나 거부됨

원인:

- VRF 키 설정 문제
- 제출 타이밍 문제
- 가스 부족

해결 방법:

1. VRF 키 설정 확인

```
oasis-node identity show-vrf-key
```

2. VRF 상태 확인

```
oasis-node beacon show-vrf-state
```

3. 가스 설정 조정

```
beacon:
vrf_parameters:
gas_costs:
vrf_prove: 2000  # 가스 한도 증가
```

문제 3: 랜덤 비콘 값이 동일함

증상: 여러 에포크에서 동일한 비콘 값 반환

원인:

- Insecure 백엔드 사용 (개발/테스트 환경)
- VRF 증명 부족
- 비콘 계산 오류

해결 방법:

1. 백엔드 설정 확인

```
oasis-node beacon show-parameters | grep  backend
```

2. VRF 증명 상태 확인
3. 프로덕션 환경에서는 VRF 백엔드 사용

로그 분석

```
# 에포크 전환 로그 분석
grep  "epoch transition"  /var/log/oasis/node.log | tail  -10
```

```
# VRF 증명 제출 로그 확인
grep "VRF proof" /var/log/oasis/node.log

# 비콘 생성 로그 확인
grep "beacon.*generated" /var/log/oasis/node.log

# 에러 로그 분석
grep "beacon.*ERROR" /var/log/oasis/node.log
```

성능 튜닝

VRF 최적화

```
beacon:
vrf_parameters:
proof_delay: 5  # 증명 지연 감소
submission_window: 50  # 제출 윈도우 축소
```

캐시 최적화

```
// 에포크 캐시 크기 조정
const epochCacheCapacity = 256  // 기본값: 128
```

참조

관련 문서

- 합의 프로세스
- 스케줄러 서비스
- 스테이킹 서비스
- 거버넌스 서비스
- 아키텍처 개요

소스 코드

- 비콘 API
- 비콘 구현
- VRF 워커

외부 리소스

- VRF 표준 (RFC 9381)
- Verifiable Random Functions 논문
- 에포크 기반 시간 시스템

마지막 업데이트: 2024년 1월

작성자: Oasis Core 문서화 팀

검토자: 비콘 개발팀